CEG5101 Modern Computer Networking

Graduate Assistants: Ms. Poorvi Joshi (e1144005@u.nus.edu), Ms. Le Minh Chau (le_minhchau@u.nus.edu), Ms. Theepana Govintharajah (theepana@u.nus.edu)
Prepared by: Mr. Vijay Narasimha
Lecturer: Assoc. Prof. Mohan Gurusamy
ECE Department, CDE., National University of Singapore

Laboratory 05

Ryu Controller as a Firewall with HTTP/ICMP Blocking

LEARNING OBJECTIVES

The Firewall Controller is an advanced SDN application that combines learning switch functionality with security policies to block specific types of traffic from designated hosts. The main features of the firewall are:

- Blocks ICMP traffic (ping) from specified source IPs
- Blocks HTTP traffic (port 80) from specified source IPs
- Maintains IP-to-MAC mapping for accurate filtering
- Drops blocked packets at controller level

```python
def __init__(self, *args, **kwargs):
    super(LearningSwitchWithFirewall13, self).__init__(*args, **kwargs)
    # MAC address table: MAC -> (switch_id, port)
    self.mac_to_port = {}

    # Firewall rules: Block HTTP and ICMP from these IPs
    self.blocked_ips = ['10.0.0.2', '10.0.0.3']

    # IP to MAC mapping for firewall enforcement
    self.ip_to_mac = {}
```

```python
def should_block_packet(self, pkt):
    """Check if the packet should be blocked based on firewall rules"""
    # Extract IP packet
    ip_pkt = pkt.get_protocol(ipv4.ipv4)
    if not ip_pkt:
        return False

    # Check if source IP is in blocked list
    if ip_pkt.src not in self.blocked_ips:
        return False

    # Check for HTTP traffic (TCP port 80)
    tcp_pkt = pkt.get_protocol(tcp.tcp)
    if tcp_pkt and (tcp_pkt.dst_port == 80 or tcp_pkt.src_port == 80):
        self.logger.info("Blocking HTTP traffic from %s", ip_pkt.src)
        return True

    # Check for ICMP traffic
    icmp_pkt = pkt.get_protocol(icmp.icmp)
    if icmp_pkt:
        self.logger.info("Blocking ICMP traffic from %s", ip_pkt.src)
        return True

    return False
```

```
# Check if packet should be blocked by firewall
if self.should_block_packet(pkt):
    self.logger.info("Dropping packet due to firewall rule")
    return  # Drop the packet
```

DISCUSSION

Note: All files required for the exercises are under the provide **ceg_code** folder.

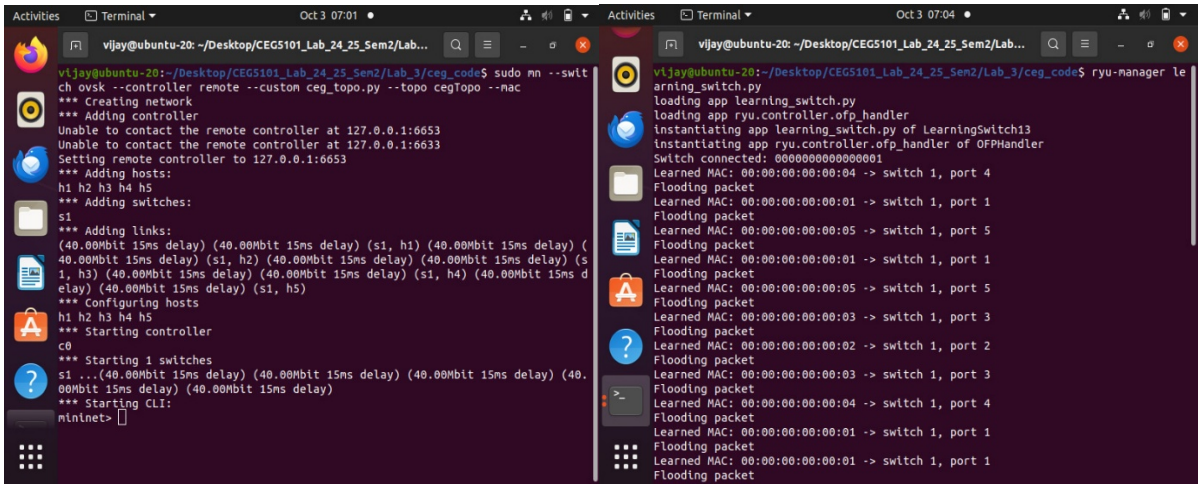
Task 1: Invoke the Ryu controller as a firewall with HTTP/ICMP blocking

1. In this activity, we will start Ryu as a firewall controller that blocks specific traffic types.

2. We will start by issuing the command to bring up the Ryu controller: "`ryu-manager firewall_http.py`".

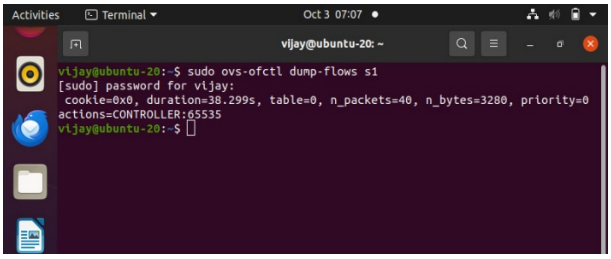
3. We will then create a topology in mininet by issuing the command "`sudo mn --switch ovsk --controller remote --custom ceg_topo.py --topo cegTopo`".

4. In the controller terminal, we observe initial handshake and switch connection messages. The firewall rules are loaded with default blocked IPs: ['10.0.0.2', '10.0.0.3'].

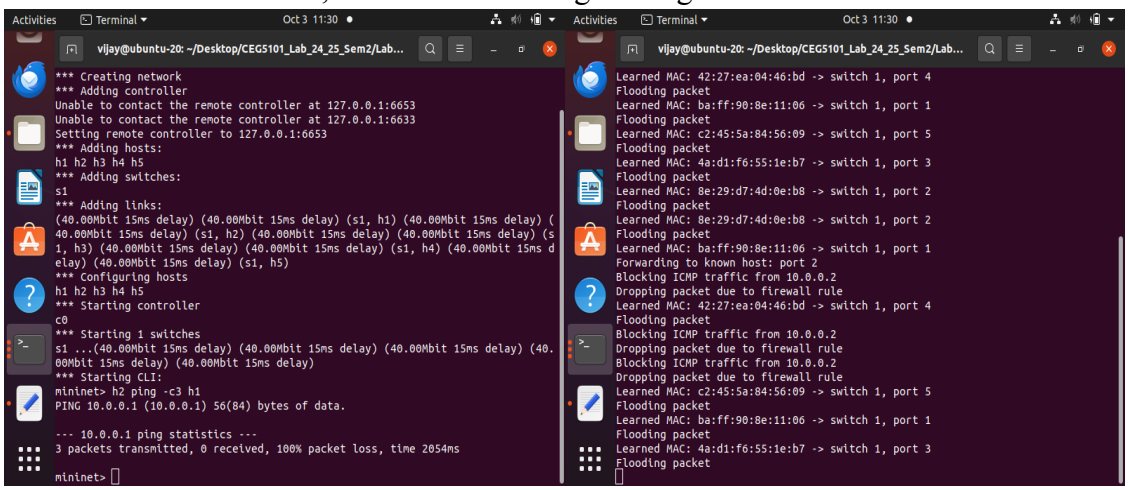


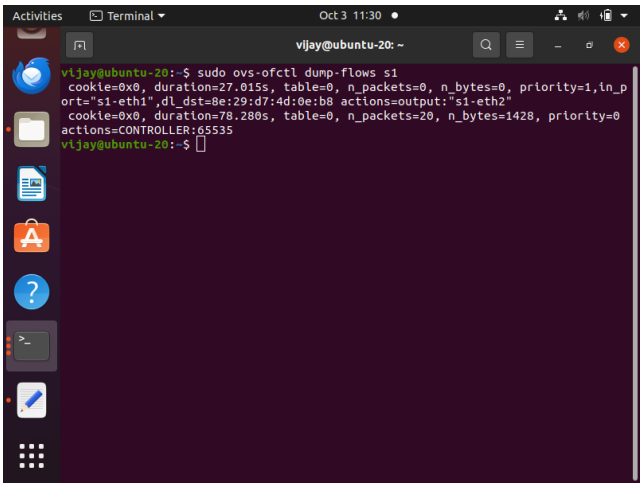
Task 2: Test ICMP Blocking Functionality

1. To check the initial flow table, we will issue the command "`sudo ovs-ofctl dump-flows s1`". We observe that only the flow entry for the controller is present.

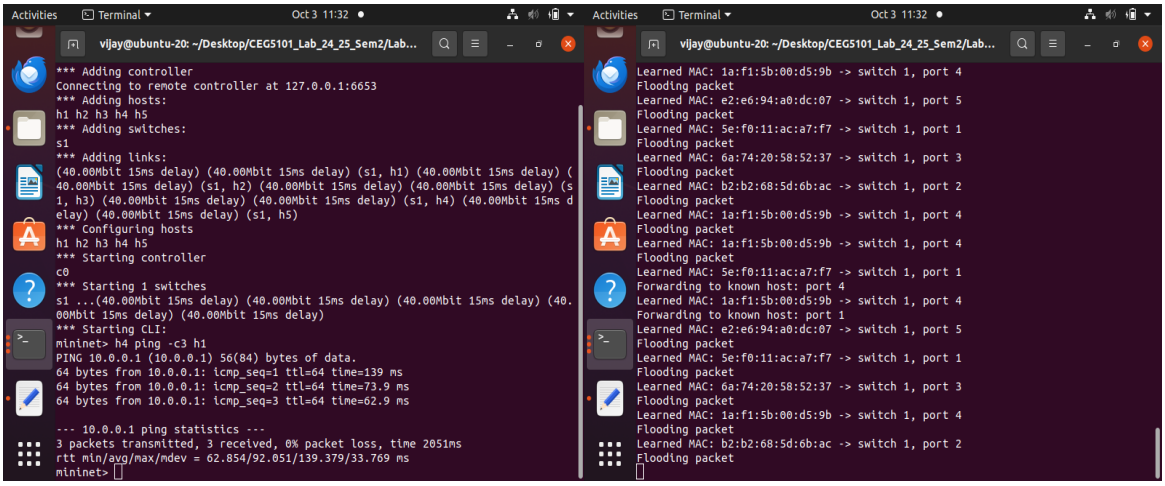


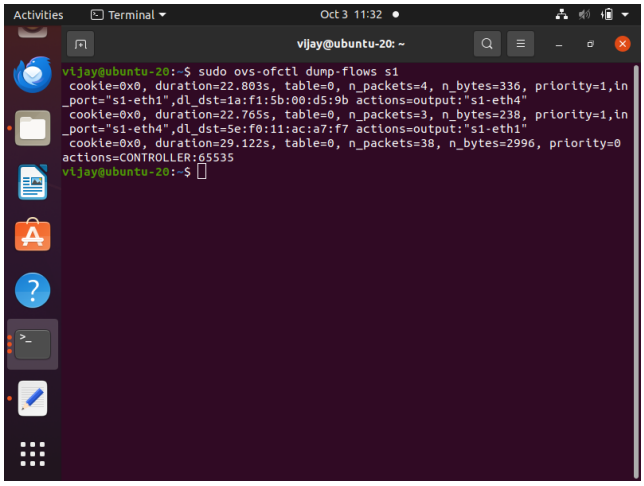
2. In Mininet CLI, we run "`h2 ping -c3 h1`" and "`h3 ping -c3 h4`". In the controller terminal, we see the blocking messages.





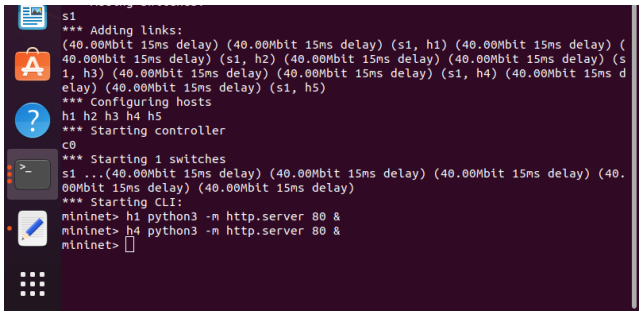
3. In Mininet CLI, we run "`h4 ping -c3 h1`" and "`h5 ping -c3 h4`". These pings succeed normally, and the controller will learn MAC addresses and install flow rules.



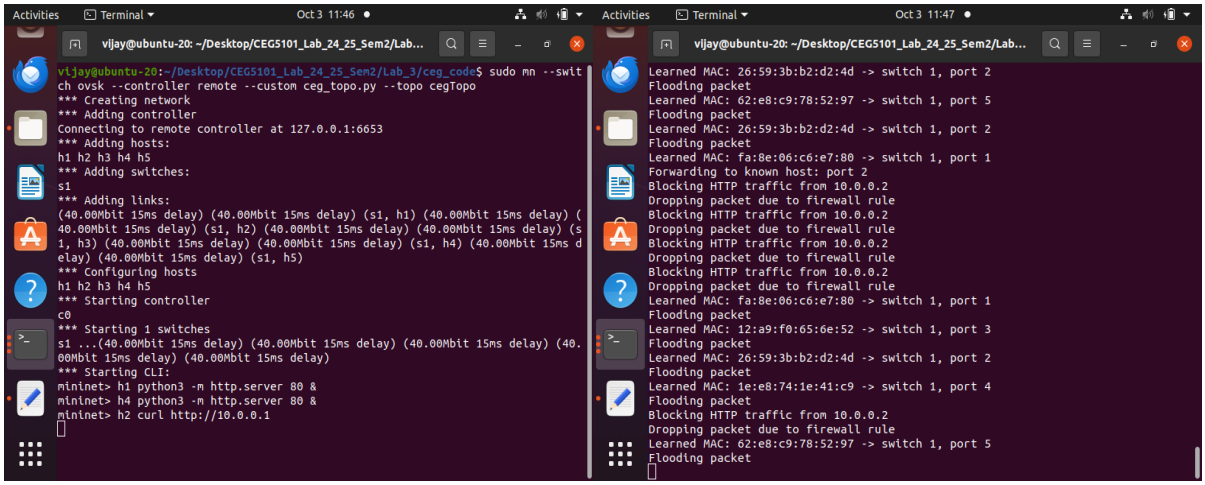


Task 3: Test HTTP Blocking Functionality

1. Start HTTP servers on port 80 of multiple hosts using the commands "`h1 python3 -m http.server 80 &`" and "`h4 python3 -m http.server 80 &`".



2. In Mininet CLI, we run "`h2 curl http://10.0.0.1`" and "`h3 curl http://10.0.0.4`". In the controller terminal, we see the blocking messages.



3. In Mininet CLI, we run "`h4 curl http://10.0.0.1`" and "`h5 curl http://10.0.0.4`". These curl commands succeed normally, and the controller will learn MAC addresses and install flow rules.

